AgriTech - Empowering farmers through digital technology

Introduction:

This AgriTech web application is a smart and scalable solution aimed at empowering farmers with technology. It integrates modern tools like weather forecasting, soil analysis, and pest alerts to streamline farm management. Designed with user-friendliness and data-driven decisions in mind, it bridges the gap between traditional farming and modern technology.

The platform is tailored to address real-world agricultural challenges using accessible and intuitive digital tools. It helps farmers increase crop yield, reduce loss, and adapt to changing environmental conditions. The app is responsive and multilingual, ensuring accessibility across different regions and literacy levels. By combining technology and farming expertise, this solution paves the way for a digitally connected agricultural ecosystem.

Description:

The application simplifies the life of farmers by offering features such as a farm dashboard, crop monitoring, weather updates, and access to expert advice. With modules for resource purchase, community forums, and data analytics, the app creates a full ecosystem for agricultural productivity. The backend is developed using the MVC pattern for scalability and maintainability. Admins manage system content and user activities, while farmers benefit from personalized features like real-time alerts, field tracking, and resource planning.

Each user type — farmers and admins — has a dedicated interface to streamline their workflows and responsibilities. The system stores and processes large volumes of agricultural data, helping users make evidence-based decisions. Security and role-based access ensure safe usage and data protection. Overall, the platform aims to digitally transform farming operations while fostering community collaboration and knowledge sharing.

Scenario Based Case Study:

Background: Ravi is a farmer in a rural area growing various crops including rice, wheat, and vegetables. He faces challenges in managing his farm efficiently due to unpredictable weather, pest infestations, and inefficient resource use. Ravi has limited access to agricultural experts and lacks precise data on his farm's conditions.

Problem: Ravi struggles with poor crop yields due to inconsistent irrigation, unpredicted weather changes, and pest attacks. He also finds it challenging to track his farm activities and manage resources efficiently.

Solution: Ravi starts using the AgriTech web application, which offers him a suite of tools to manage his farm effectively.

Implementation:

1. **Farm Setup:**
    - Ravi sets up his farm profile on the app, adding details about the size of his fields, types of crops, and irrigation methods.
2. **Weather Monitoring:**
    - The app provides real-time weather updates and forecasts. Ravi uses this feature to plan his irrigation schedule and protect his crops from adverse weather conditions.
3. **Soil and Crop Management:**
    - The app includes soil health monitoring tools that help Ravi analyze soil moisture and nutrient levels. Based on the analysis, the app suggests the optimal amount of water and fertilizers needed for his crops.
4. **Pest and Disease Alerts:**
    - Ravi receives alerts about potential pest infestations and disease outbreaks. The app suggests preventive measures and appropriate treatments.
5. **Resource Management:**
    - Ravi tracks his farm activities such as planting, fertilizing, and harvesting through the app. This helps him manage his resources more efficiently and keep detailed records.
6. **Expert Consultation:**
    - The app provides access to agricultural experts who can offer advice and answer Ravi's questions. He can also participate in community forums to share experiences with other farmers.

Outcome: With the help of the AgriTech web application, Ravi significantly improves his crop yield. He efficiently manages his resources, reduces losses due to pests and diseases, and makes informed decisions based on accurate weather forecasts and soil health data. The overall productivity and profitability of his farm increase.

SYSTEM REQUIREMENTS:

To ensure smooth development, deployment, and usage of the AgriTech Web Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable agricultural platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Node.js (v16 or above)**: Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 👉 Download Node.js
- **npm (v8 or above)**: A package manager required to install dependencies for React and related libraries.
- **React.js**: A JavaScript library for building dynamic and responsive user interfaces.
- **Browser**: Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Express.js**: A lightweight web framework for building RESTful APIs.
- **MongoDB**: A NoSQL database used to store structured and unstructured data related to farms, users, and resources. 👉 Download MongoDB
- **Postman**: Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.
- **Git & GitHub**: For version control and collaborative development.

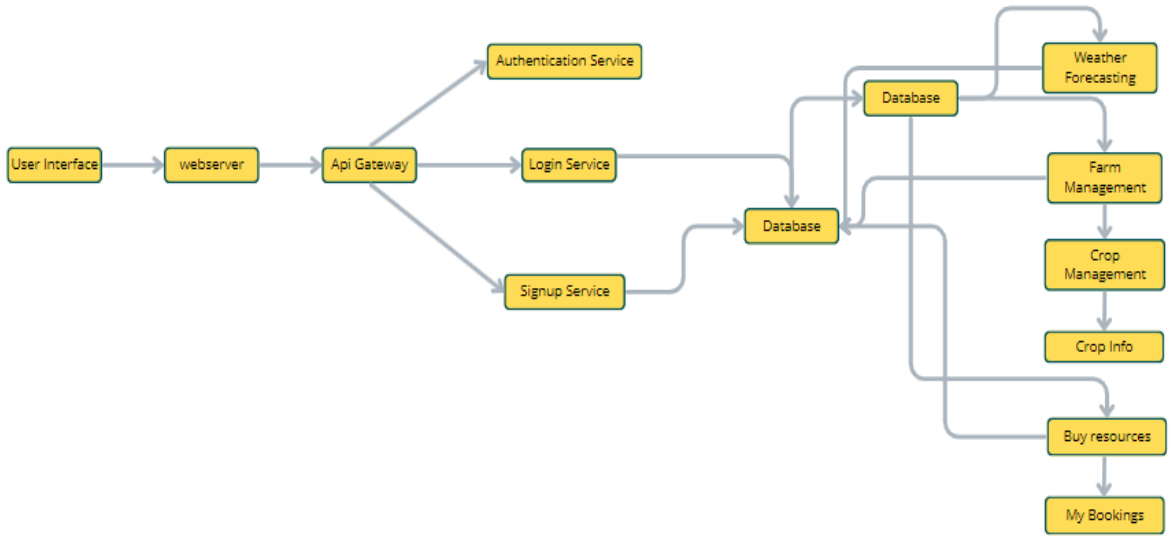
2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM**: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage**: At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.
- **Display**: 1366x768 or higher — Recommended for optimal coding experience and application layout visualization.

PROJECT ARCHITECTURE:

1) Technical Architecture:

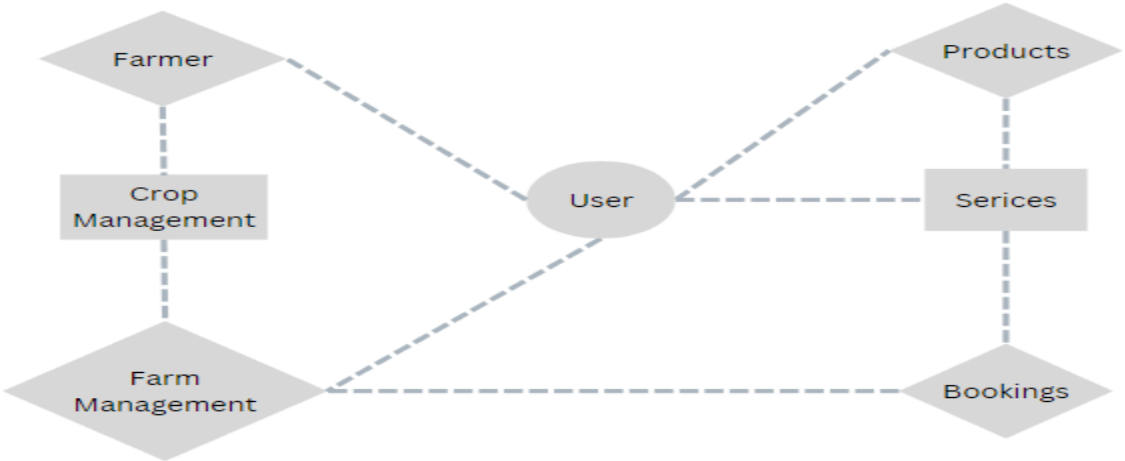


In this technical architecture, the AgriTech consists of several components:

- User Interface: The web interface provides an intuitive platform for farmers to access and interact with the AgriTech system. Here, farmers can explore crop information, manage farm activities, and make purchases for agricultural resources, ensuring a seamless user experience.

- Web Server: Hosts the user interface and serves the web pages to the users.
- API Gateway: Acts as a single entry point for client requests and routes them to the respective services.
- Authentication Service: Handles user authentication and authorization for secure access to the system.
- Database: Stores persistent data related to Crop Management, Farm Management and other entities crucial to the AgriTech Web App.
- Weather Forecasting: Receive real-time weather updates and forecasts tailored to your farm's location, helping you plan your activities and protect your crops from adverse weather conditions.
- Farm Management: Easily manage your farm activities, track crop growth stages, and view historical data from a centralized dashboard, ensuring efficient farm operation.
- Buy Resources: Purchase agricultural resources such as seeds, fertilizers, and pesticides directly through the app, saving you time and effort.
- Crop Management: Keep track of your crops' progress, monitor soil health, and receive recommendations for optimal care, maximizing your yield potential.
- Crop Info: Access detailed information about different crops, including planting guides, care instructions, and pest management tips, helping you make informed decisions about your crops.
- My Orders: View and manage your bookings for farm-related services, such as equipment rentals or expert consultations, ensuring timely and efficient service delivery.

2)ER-DIAGRAM:



User:
UserID (PK)
Name
Email
Phone
Address

Farm:
FarmID (PK)
FarmName
FarmSize

Crop:
CropID (PK)
CropName (FK)
CropType
SowingTime
HarvestingTime
Booking:
BookingID (PK)
UserID (FK)
ProductID (FK)
BookingDate
TotalAmount

3)Key Features:

1. **Farm Management Dashboard:** A centralized dashboard to monitor and manage farm activities, track crop growth stages, and view historical data.
2. **Weather Forecasting:** Real-time weather updates and forecasts tailored to the farm's location, helping farmers plan their activities accordingly.
3. **Soil Health Monitoring:** Tools to analyze soil moisture and nutrient levels, providing recommendations for optimal crop growth.
4. **Pest and Disease Alerts:** Alerts and notifications about potential pest infestations and disease outbreaks with suggested treatments.
5. **Resource Management:** Features to track and manage the use of water, fertilizers, and other resources, ensuring efficient utilization.

6. **Expert Consultation:** Access to agricultural experts for personalized advice and solutions to farming challenges.
7. **Community Forum:** A platform for farmers to share experiences, ask questions, and learn from each other.
8. **Crop and Field Tracking:** Detailed tracking of crop growth stages, field conditions, and yield predictions.
9. **Data Analytics and Reports:** Analytical tools to generate reports on farm performance, resource usage, and crop yields, helping farmers make data-driven decisions.

4) Roles and Responsibility

Farmer:

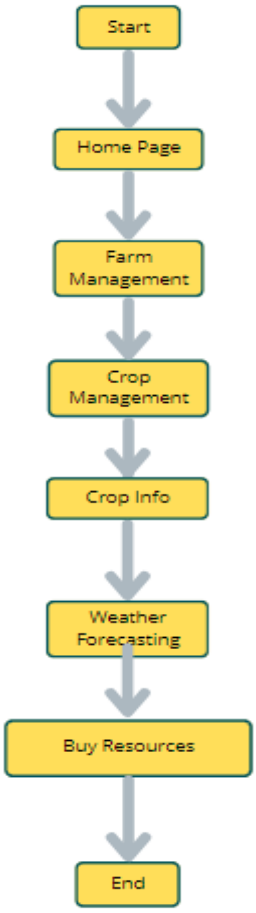
- **Crop Management:**
  - Monitor and manage crop growth stages.
  - Record and analyze soil health data.
  - Receive and implement recommendations for optimal crop care.
- **Farm Management:**
  - Set up and maintain farm profiles.
  - Track farm activities such as planting, fertilizing, and harvesting.
  - Optimize resource usage, including water, fertilizers, and pesticides.
- **Weather Monitoring:**
  - Access real-time weather updates and forecasts.
  - Plan farming activities based on weather conditions.
- **Pest and Disease Management:**
  - Receive alerts about potential pest infestations and disease outbreaks.
  - Implement preventive measures and treatments.
- **Purchasing Resources:**
  - Buy agricultural products such as seeds, fertilizers, and pesticides.
- **Booking Services:**
  - Schedule and manage bookings for services like equipment rentals and expert consultations.
- **Community Engagement:**
  - Participate in community forums to share experiences and learn from other farmers.

Admin:

- **User Management:**
  - Manage user registrations and profiles.

    - Verify and approve new user accounts.
    - Handle user queries and support requests.
- **Content Management:**
    - Update and manage information related to crops, products, and services.
    - Ensure the accuracy and relevance of content available to users.
- **Resource Management:**
    - Monitor and update the inventory of agricultural products.
    - Coordinate with suppliers to ensure the availability of resources.
- **Service Management:**
    - Oversee the scheduling and management of services offered through the application.
    - Ensure timely and efficient delivery of booked services.
- **System Maintenance:**
    - Perform regular system checks and maintenance.
    - Ensure the smooth functioning of the application.
    - Troubleshoot and resolve technical issues.
- **Security Management:**
    - Implement security measures to protect user data.
    - Monitor and address any security threats or breaches.
- **Data Analytics:**
    - Analyze user data to gain insights into user behavior and needs.
    - Generate reports to assist in strategic planning and decision-making.
- **Policy Enforcement:**
    - Ensure compliance with application policies and terms of use.
    - Handle violations and take appropriate actions.

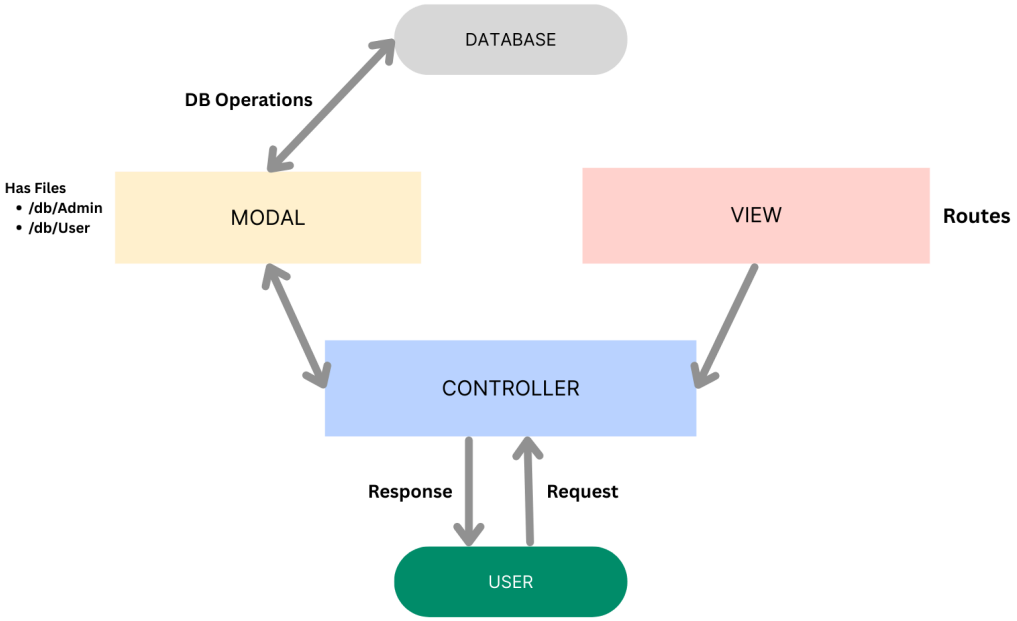
5) User Flow:



1. **Start:** This is the entry point of the user flow.
2. **Home Page:** Users arrive at the home page of the AgriTech web application. This page likely includes navigation options to various sections such as farm management, crop management, weather forecasting, and resources.
3. **Farm Management:** Users navigate to the farm management section where they can add, view, edit, or delete information about their farms. This includes details like farm location, size, and other relevant data.
4. **Crop Management:** After managing farms, users move to crop management. Here, they can add, view, edit, or delete information about the crops grown on their farms. This section includes details like crop type, planting dates, and expected harvest dates.
5. **Crop Info:** Users access detailed information about their crops. This might include growth stages, health status, pest control measures, and more. It provides a comprehensive view of each crop's status.

6. **Weather Forecasting:** Users navigate to the weather forecasting section to get current weather updates and forecasts for their farm locations. This helps them make informed decisions about farming activities based on weather conditions.
7. **Buy Resources:** Users proceed to buy resources needed for farming, such as seeds, fertilizers, pesticides, and equipment. This section likely includes a marketplace where users can browse and purchase these resources.
8. **End:** This is the end point of the user flow, signifying the completion of the main tasks a user needs to perform within the AgriTech application.

6)MVC PATTERN:



The AgriTech backend application follows the **Model-View-Controller (MVC)** architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.

Model Layer (Data Layer)

The **Model** layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using **Mongoose**, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The **Controller** layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the **View** is implemented as the **routing layer**, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- **Separation of Concerns**: Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability**: New features can be added easily by creating new routes, controllers, and models.
- **Reusability**: Logic in controllers and models can be reused across multiple parts of the application.
- **Testing**: Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly**: Multiple developers can work simultaneously on different layers without conflict.

This EPIC Journey includes 6 Steps :

This project follows a full-stack architecture, comprising a frontend built using **React (Vite)** and a backend built using **Node.js with Express**. It is structured for scalability, modularity, and performance, with a focus on clean code separation between the user interface and server-side logic.

EPIC 1: Project Setup and Configuration:

This section outlines the necessary steps to set up the development environment for the AgriTech web application, including installation of tools, folder structure, and required packages for both frontend and backend development.

1. **Create a React + Vite Project Setup**

    - Instead of Create React App, we will use **Vite** for a faster development experience.

      Create the root project folder:

      ```
      mkdir agritech
      cd agritech
      ```

      Create separate folders for Client and Server:

      ```
      mkdir Frontend
      mkdir Backend
      ```

2. **Create Project Folders and Files**

Organize the project using a clean and scalable folder structure:

- **Client Folder (Frontend)**

Contains all React code such as components, pages, routes, and styles.
Commands for Frontend setup :

npm create vite@latest client -- --template react
cd client
npm install

- **Server Folder (Backend)**

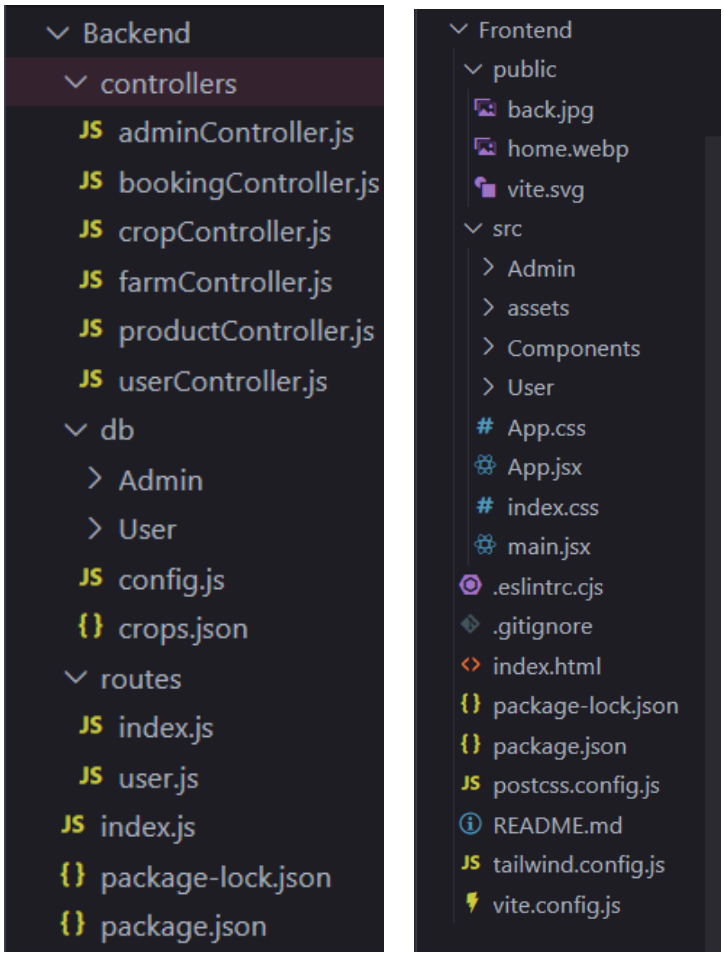
Hosts the Express server, API routes, database models, and middleware.
Commands for Backend Setup :

mkdir server

cd server

npm init -y

=> Once the separate Frontend and Backend folders are created, set up the following folder structure in each of them.



3. Install Required Packages

Once the folders are set, navigate into each and install the necessary packages.

EPIC 2: Backend Development:

Backend npm Packages

Install in the `/server` directory:

```
npm install express mongoose cors dotenv
```

- Express – Minimal web framework for creating the API server.
- Mongoose – ODM (Object Data Modeling) tool for MongoDB.
- CORS – Middleware to enable cross-origin requests between frontend and backend.
- Dotenv – Loads environment variables from a `.env` file for secure key management.
- Nodemon

=> `index.js` (Root file)

- Entry point of the backend (Express server setup).
- Configures:
    - `cors` for cross-origin requests.
    - `body-parser`/`express.json()` for parsing JSON payloads.
    - Routes imported from `routes/`.
- Loads environment variables from `.env` and starts the server.

=> `controllers/`

Each file in this folder handles business logic and interacts with Mongoose models.

- `adminController.js`:
  Manages admin-specific actions like verifying users, viewing reports, system analytics, etc.
- `bookingController.js`:
  Handles service or equipment booking-related logic — creating, updating, and viewing bookings.
- `cropController.js`:
  Logic for managing crops: adding crop details, monitoring stages, applying treatments, etc.
- `farmController.js`:
  Handles operations related to farm details like size, location, crop assignments, etc.
- `productController.js`:
  Manages inventory-like products such as seeds, fertilizers, pesticides — create, list, update.
- `userController.js`:
  Includes user registration, login, profile update, and user-specific actions.

Role: Keeps route handlers clean by offloading logic.

=> db/

Stores database connection, schema models, and static JSON data.

- `config.js`:
  Establishes MongoDB connection using Mongoose with credentials from `.env`.
- `Admin/`, `User/`:
  These may contain Mongoose model files defining schema for admin and user collections.
- `crops.json`:
  Static JSON containing crop data such as crop types, durations, pests, etc., for preloading or reference.

=> routes/

Handles HTTP routes for different parts of the app.

- `user.js`:
  Contains routes for user registration, login, logout, and user-specific endpoints.
- `index.js`:
  Main router file that aggregates and mounts all sub-routes under appropriate API paths (e.g., `/api/users`, `/api/farms`).

Role: Separates API endpoints from business logic and keeps route handling modular.

=> Authentication

- Middleware protects private routes (e.g., dashboard, crop updates).
- Authentication flow:
  - User registers or logs in.
  - Receives a token.
  - Token is sent in headers on protected requests.
  - Middleware verifies and authorizes user access.

Reference Link :-
https://drive.google.com/file/d/1YVTeDhtXSWa3kATwzVKZRg-S5aed7W2w/view?usp=drive_link

EPIC 3 : Database:

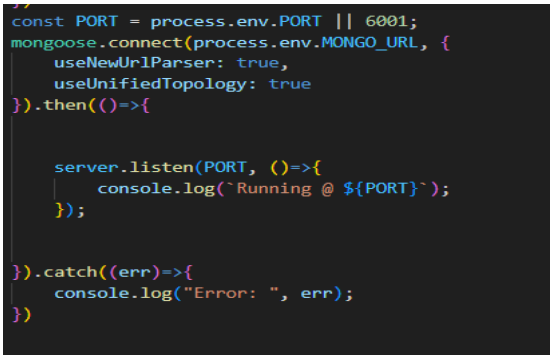
- **1. Configure MongoDB**

To manage and persist user and application data, we use **MongoDB** — a flexible, document-oriented NoSQL database — in combination with **Mongoose**, which is an elegant ODM (Object Data Modeling) library for Node.js. This setup allows seamless interaction with the MongoDB database through JavaScript/TypeScript.

- **Step 1: Install Mongoose**
- **Step 2: Create Database Connection**
- **Step 3: Create Schemas & Models**

2. Connect database to backend:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below.



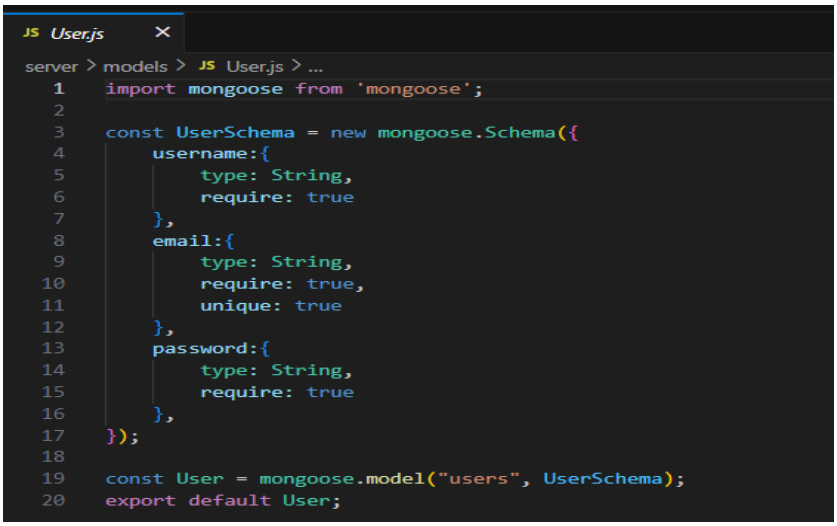
```
const PORT = process.env.PORT || 6001;
mongoose.connect(process.env.MONGO_URL, {
    useNewUrlParser: true,
    useUnifiedTopology: true
}).then(()=>{


    server.listen(PORT, ()=>{
        console.log(`Running @ ${PORT}`);
    });


}).catch((err)=>{
    console.log("Error: ", err);
})
```

3. Configure Schema:

Firstly, configure the Schemas for MongoDB database, to store the data in such a pattern. Use the data from the ER diagrams to create the schemas.
The schemas are looks like for the Application.



EPIC 4: Frontend Development:

1. Setup React Application

This step involves initializing and preparing the base environment for frontend development:

- **Create a React Application**:
  The project begins with creating a new React application using tools like `create-react-app` or `Vite` to scaffold the frontend structure. This generates essential configuration files and boilerplate code to kickstart development.
- **Configure Routing**:
  React Router (commonly used) is configured to manage client-side routing. It helps navigate between different views and screens (such as login, signup, dashboard) without reloading the page, thus creating a seamless Single Page Application (SPA) experience.
- **Install Required Libraries**:
  Additional libraries are installed to enhance development and user experience. This includes UI component libraries (like Material-UI or React Bootstrap), HTTP clients (like Axios or Fetch), form handling libraries, state management tools, and more.
- **Frontend npm Packages**

Install in the `/frontend` directory:

```
npm install axios react-router-dom bootstrap react-bootstrap
react-icons
```

- Axios – For making HTTP requests from React to the backend.
- React-Router-Dom – Enables navigation between components (pages) in a SPA.
- Bootstrap – CSS framework for responsive design.
- React-Bootstrap – Bootstrap components built specifically for React.
  React-Icons – Collection of popular icon packs for UI design.
- moment – used for handling time & date formatting
- recharts – for data visualization
- tailwindcss, vite, eslint – if you are showcasing the frontend stack fully

2. Design UI Components

In this phase, visual elements of the application are planned and implemented to ensure usability and consistency:

- **Create Components**:
  The interface is modularized by dividing it into reusable components such as buttons, forms, headers, and cards. This promotes reusability and maintainability across the application.
- **Implement Layout and Styling**:
  A responsive and visually appealing layout is created using CSS, CSS-in-JS libraries, or utility frameworks like Tailwind CSS. The design ensures compatibility across screen sizes and devices.
- **Add Navigation**:
  Logical and intuitive navigation is implemented using navbar, sidebar, or drawer components. Routing logic ensures users can move between different sections easily, maintaining context and session state as needed.

3. Implement Frontend Logic

This step focuses on making the frontend dynamic by connecting it to the backend and managing real-time user interactions:

- **Integration with API Endpoints**:
  The frontend interacts with backend services via HTTP requests. APIs handle operations such as user registration, login, data retrieval, and updates. Proper handling of responses and errors ensures data integrity and user feedback.
- **Implement Data Binding**:
  User inputs and application state are managed using React's state management system. This enables real-time updates to the UI as the user interacts with the app. Two-way binding ensures form values reflect state and vice versa, enhancing interactivity.

4. API integration:

The **Weather Forecast** component integrates real-time weather data into the application by consuming the **OpenWeatherMap API**. This public RESTful API provides detailed meteorological data based on geographic coordinates or city names. It enables the frontend to display temperature, humidity, wind speed, and other weather metrics for the user's current or searched location.

=> Getting the OpenWeatherMap API Key

To use the OpenWeatherMap API, follow these steps to obtain an API key:

1. Visit https://home.openweathermap.org/users/sign_up
2. Create a free account or log in.
3. Navigate to the **API Keys** section of your profile.
4. Generate a new API key (or use the default one provided).
5. Copy this API key — it will be used in the API endpoints mentioned below.

=> Key API Endpoints Used:

1. Get Weather by Coordinates (Auto-Detection)

```
https://api.openweathermap.org/data/2.5/weather?lat={latitude}&lon={longitude}&appid={API_KEY}&units=metric
```

- Called immediately upon component mount.

- Uses the browser's **Geolocation API** to retrieve the user's current latitude and longitude.
- These coordinates are passed to the endpoint to fetch weather data for the current physical location.

2. Get Weather by City Name (Manual Search)

`https://api.openweathermap.org/data/2.5/weather?q={city_name}&appid={API_KEY}&units=metric`

- Used when a user manually types a city name and presses the **Search** button or hits **Enter**.
- Fetches and displays weather data for the searched city.

=> Frontend Integration Flow

Automatic Location Detection:

- Use `navigator.geolocation.getCurrentPosition()` to get the user's coordinates.
- Pass these coordinates to the OpenWeatherMap API endpoint.
- Parse and display the response:
  - Temperature
  - Humidity
  - Wind Speed
  - Atmospheric Pressure
  - Weather Description
  - Weather Icon (based on condition)

Manual Location Search:

- The user enters a city name in the search input field.
- On clicking the "Search" button:
  - Make an API call using the entered city name.
  - Retrieve and display the respective weather information on the UI.

Referance Link :-
https://drive.google.com/file/d/1MTNVY_8NcQ42mz_XS8sPu5ZHn0RlBFbz/view?usp=drive_link

EPIC 5: Project Implementation:

Run the Complete Application and start backend and frontend servers:

Backend

```
cd Backend
npm start
```

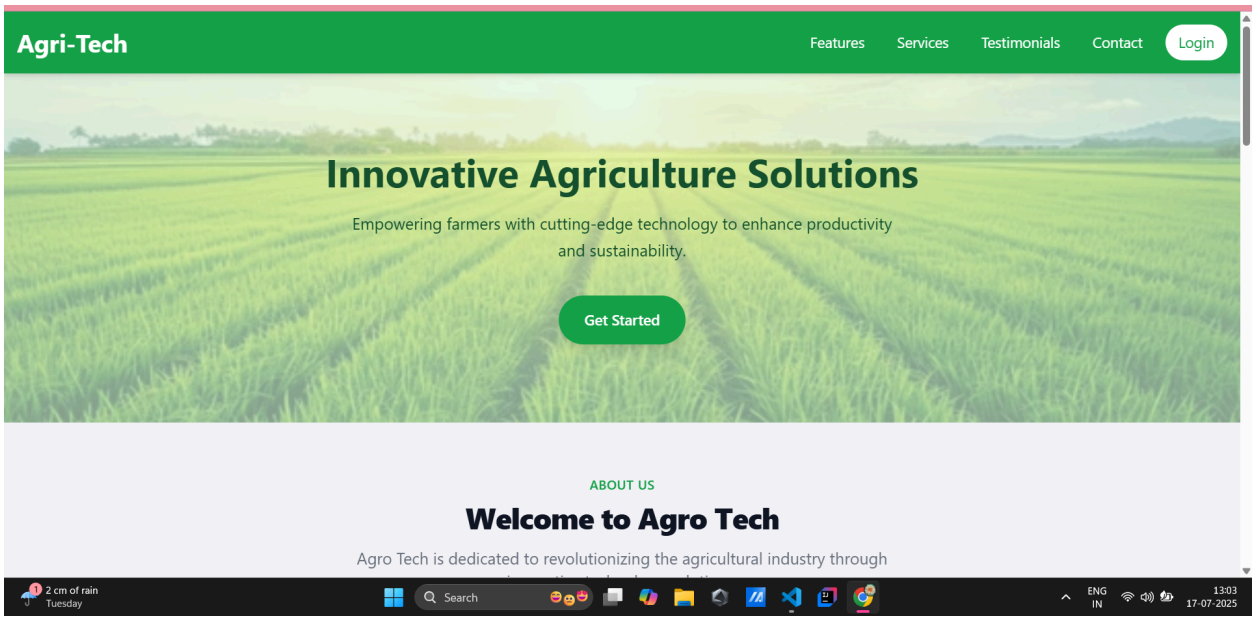
Frontend

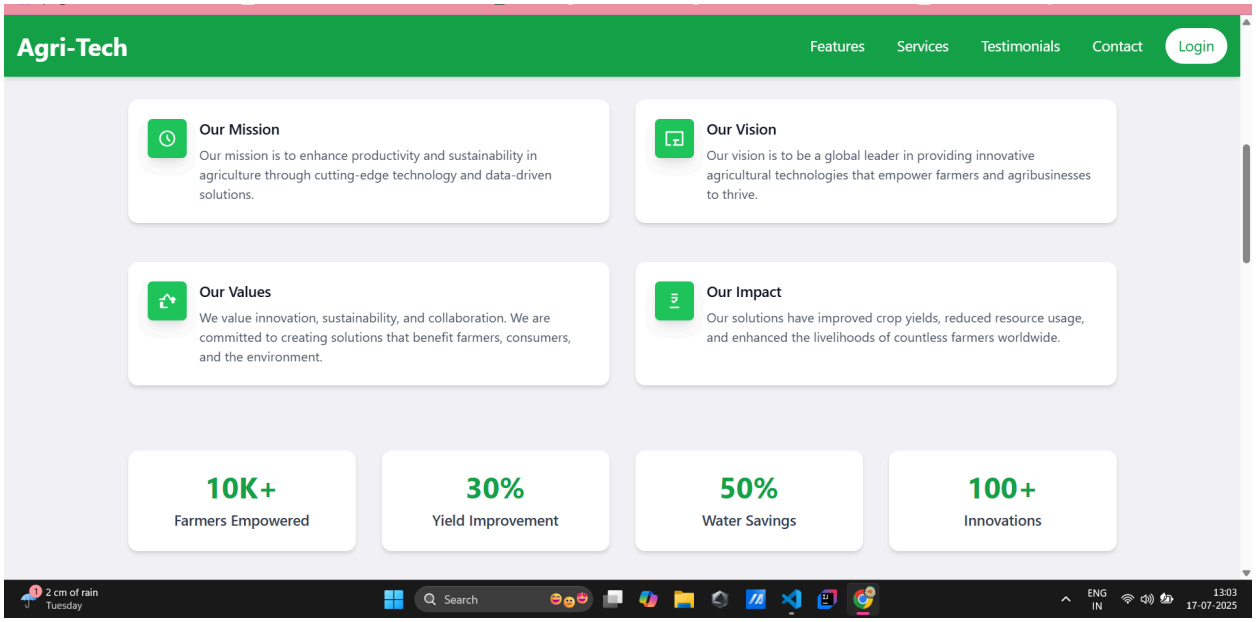
```
cd Frontend
npm run dev
```

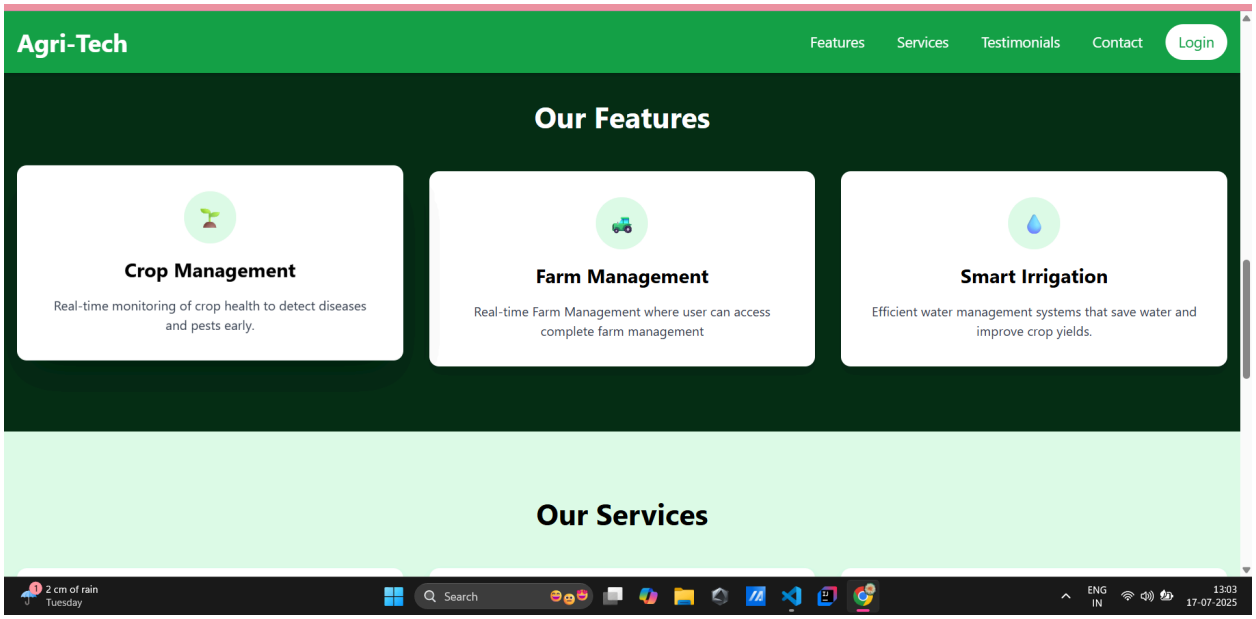
Project Screenshots:

Finally, after finishing coding the projects we run the whole project to test it's working process and look for bugs. Now, let's have a final look at the working of our AgriTech.

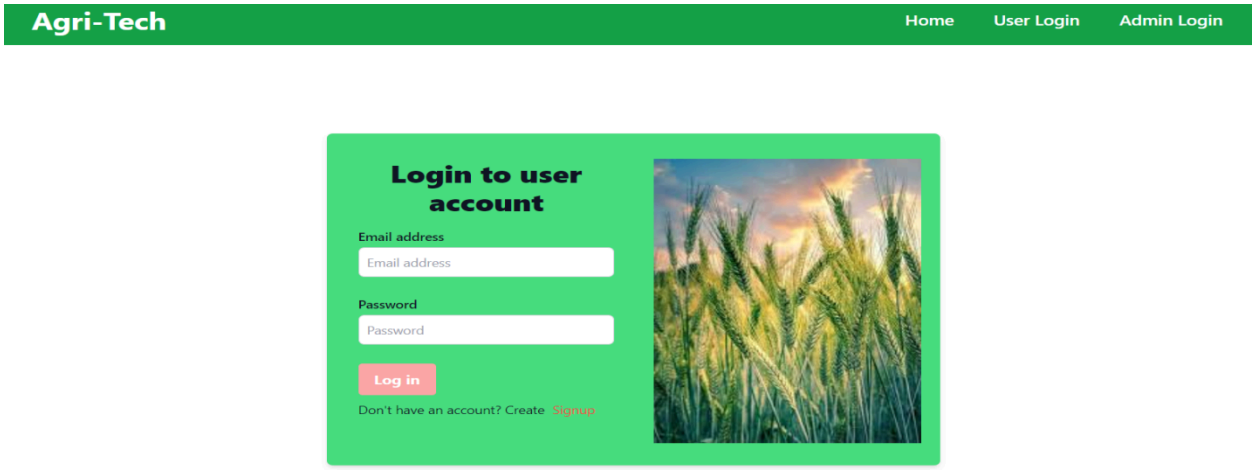
Landing page:-



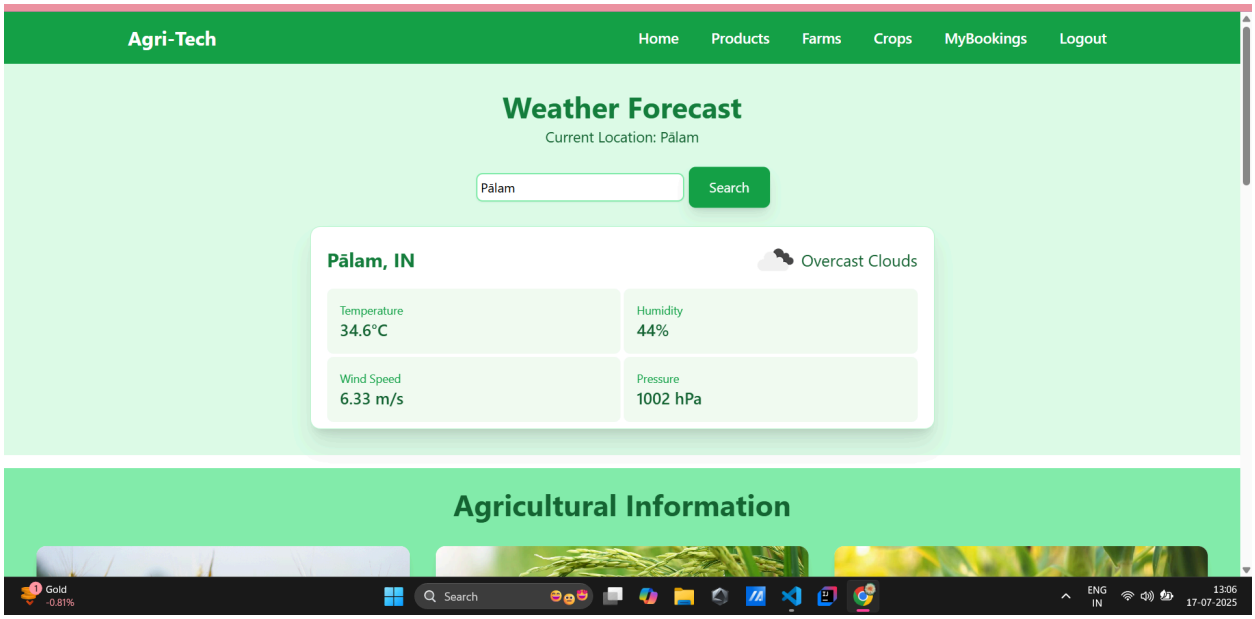

Agri-Tech
Features
Services
Testimonials
Contact
Login
Our Mission
Our mission is to enhance productivity and sustainability in agriculture through cutting-edge technology and data-driven solutions.
Our Vision
Our vision is to be a global leader in providing innovative agricultural technologies that empower farmers and agribusinesses to thrive.
Our Values
We value innovation, sustainability, and collaboration. We are committed to creating solutions that benefit farmers, consumers, and the environment.
Our Impact
Our solutions have improved crop yields, reduced resource usage, and enhanced the livelihoods of countless farmers worldwide.
10K+
Farmers Empowered
30%
Yield Improvement
50%
Water Savings
100+
Innovations
2 cm of rain
Tuesday
Search
ENG
IN
13:03
17-07-2025

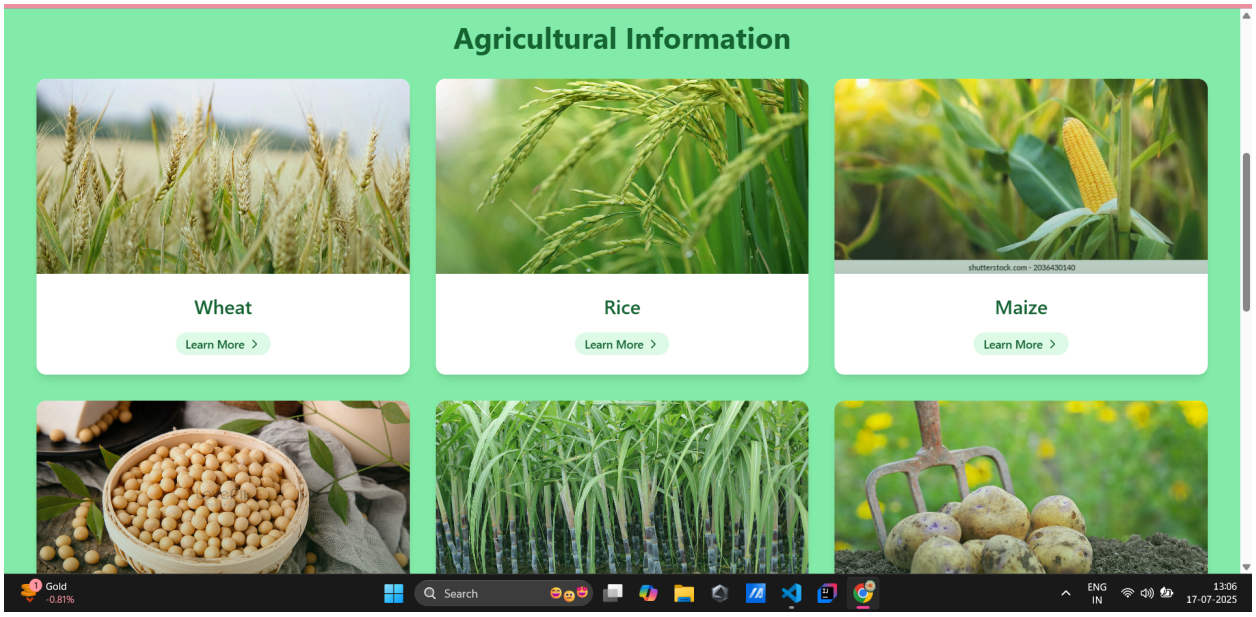

Agri-Tech
Features
Services
Testimonials
Contact
Login
Our Features
Crop Management
Real-time monitoring of crop health to detect diseases and pests early.
Farm Management
Real-time Farm Management where user can access complete farm management
Smart Irrigation
Efficient water management systems that save water and improve crop yields.
Our Services
2 cm of rain
Tuesday
Search
ENG
IN
13:03
17-07-2025

Login Page:-

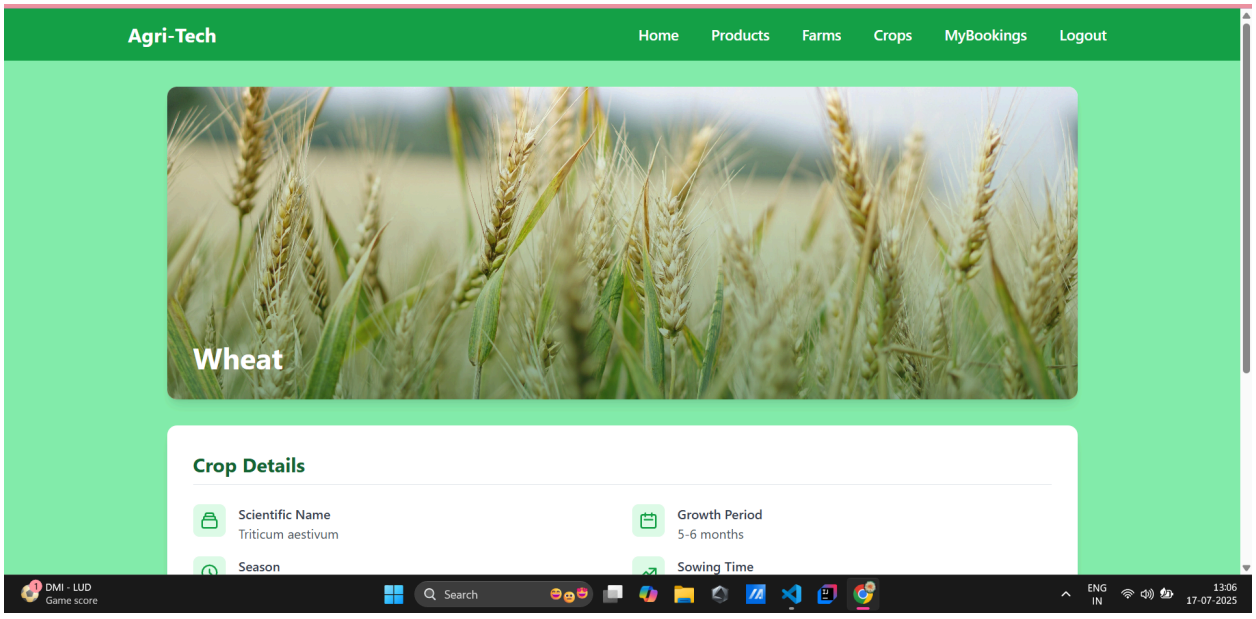


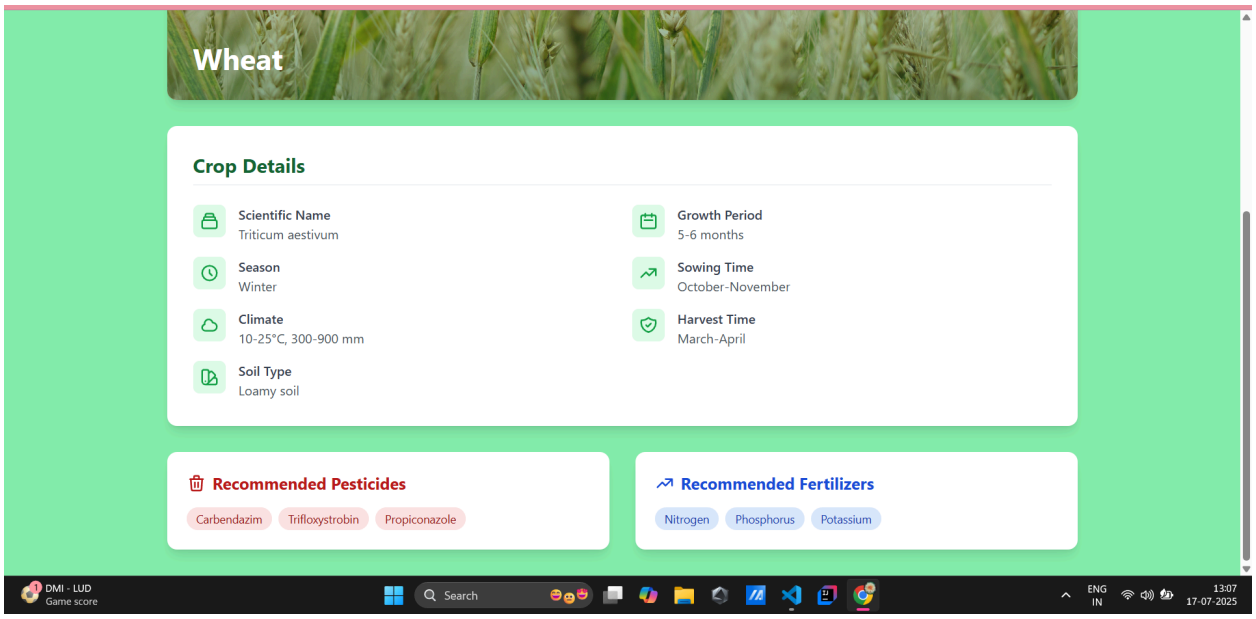
User Home Page:-



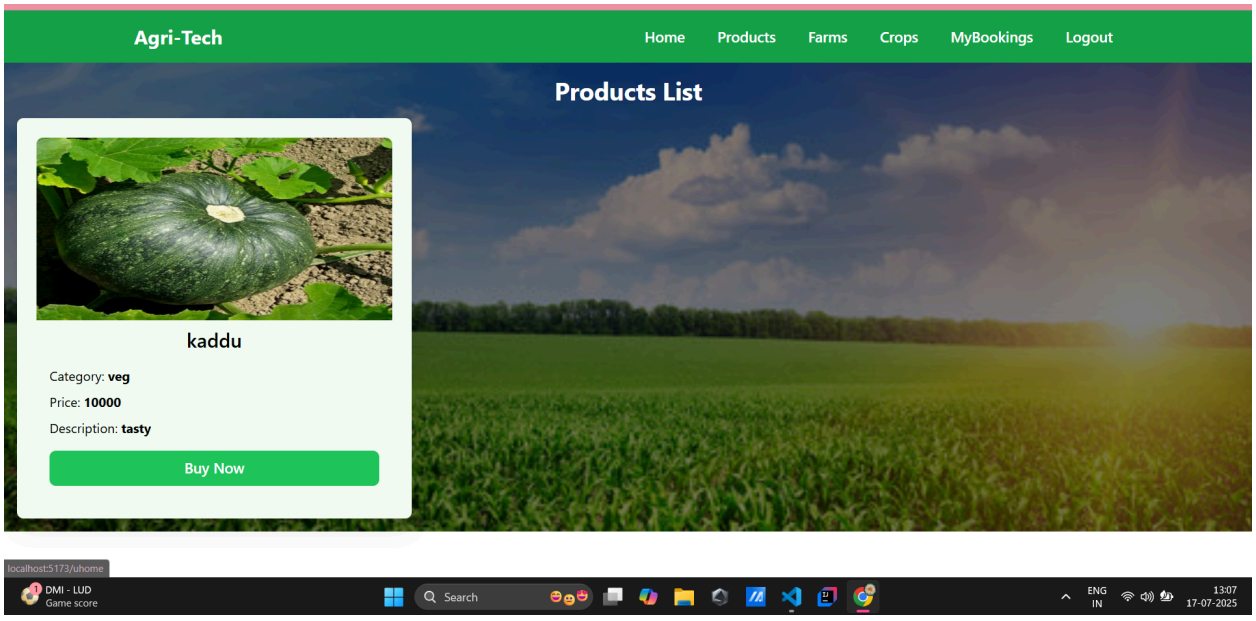


Crop Information:

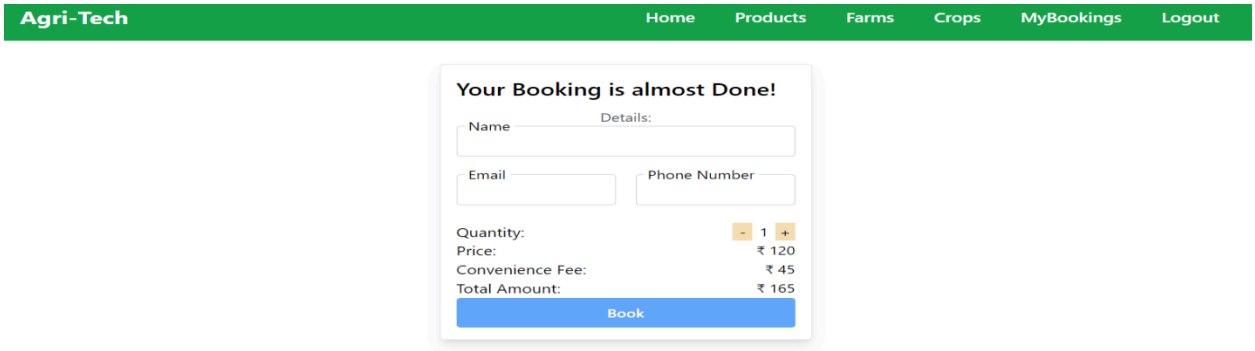




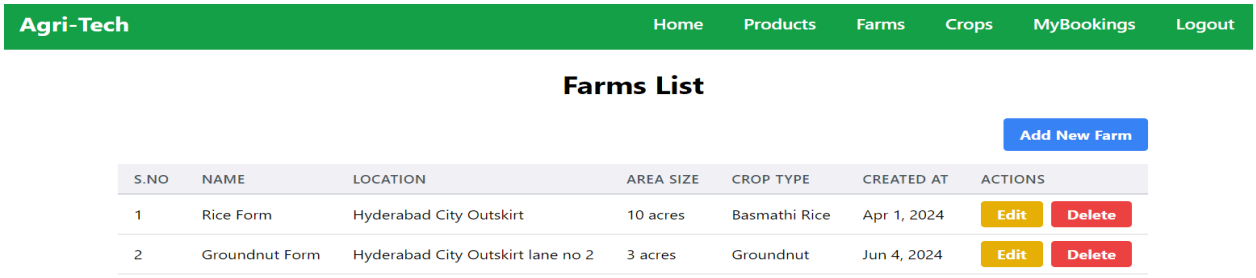
Products List:



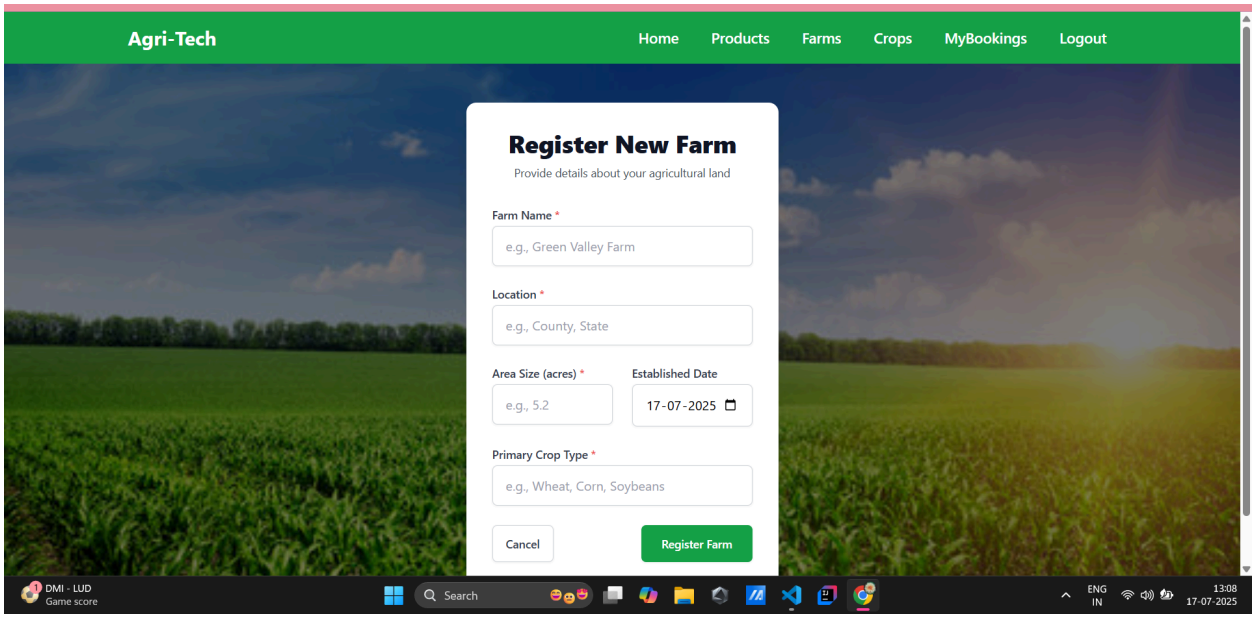
Order Product:



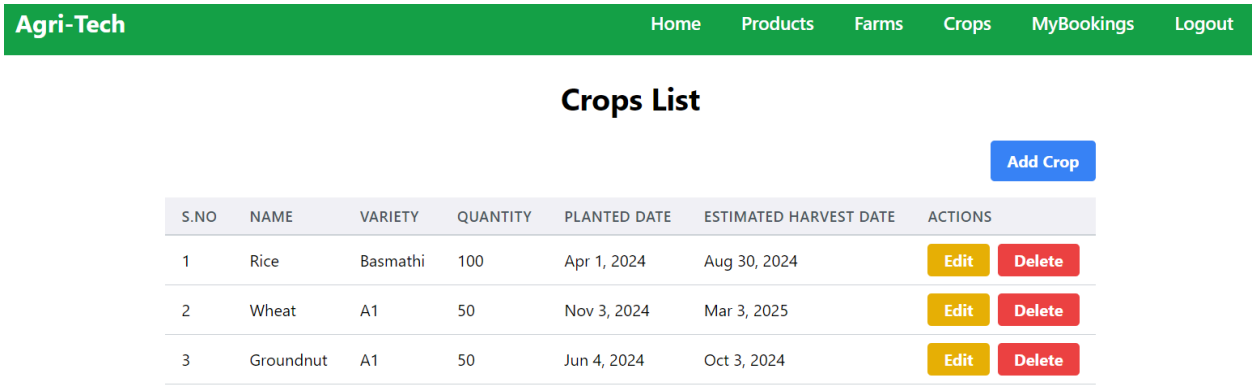
Farms:



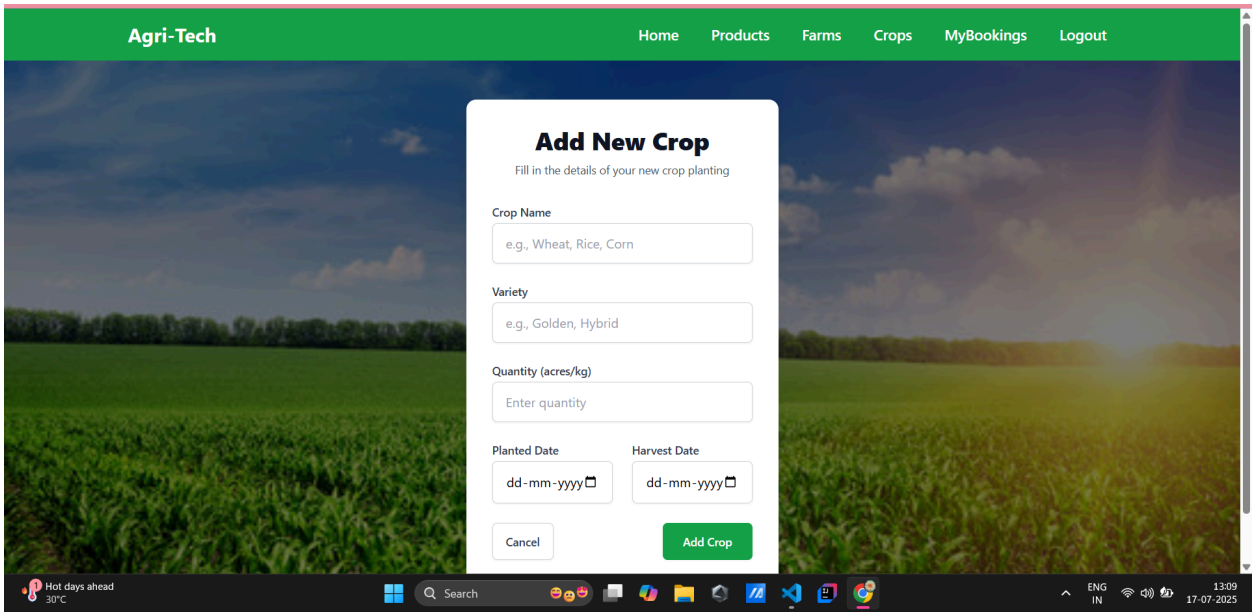
Add Farm:



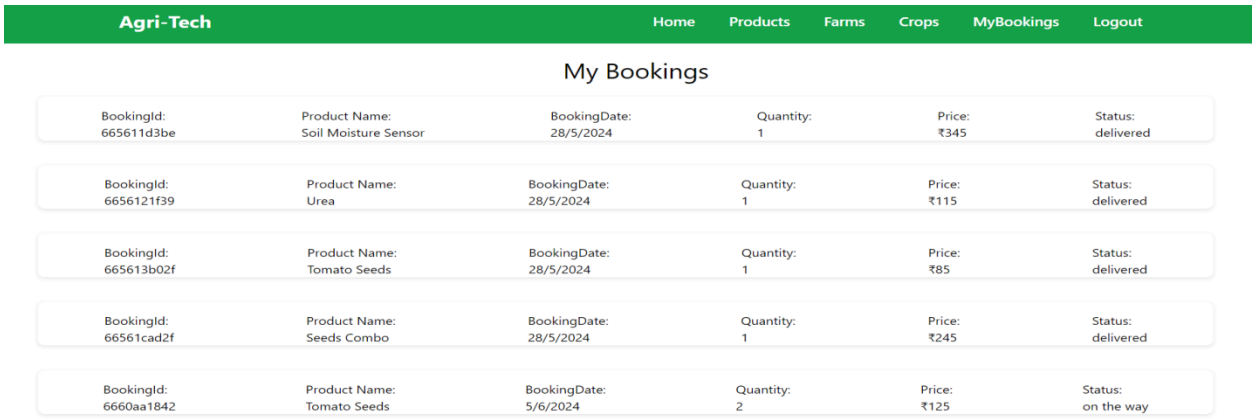
Crops:



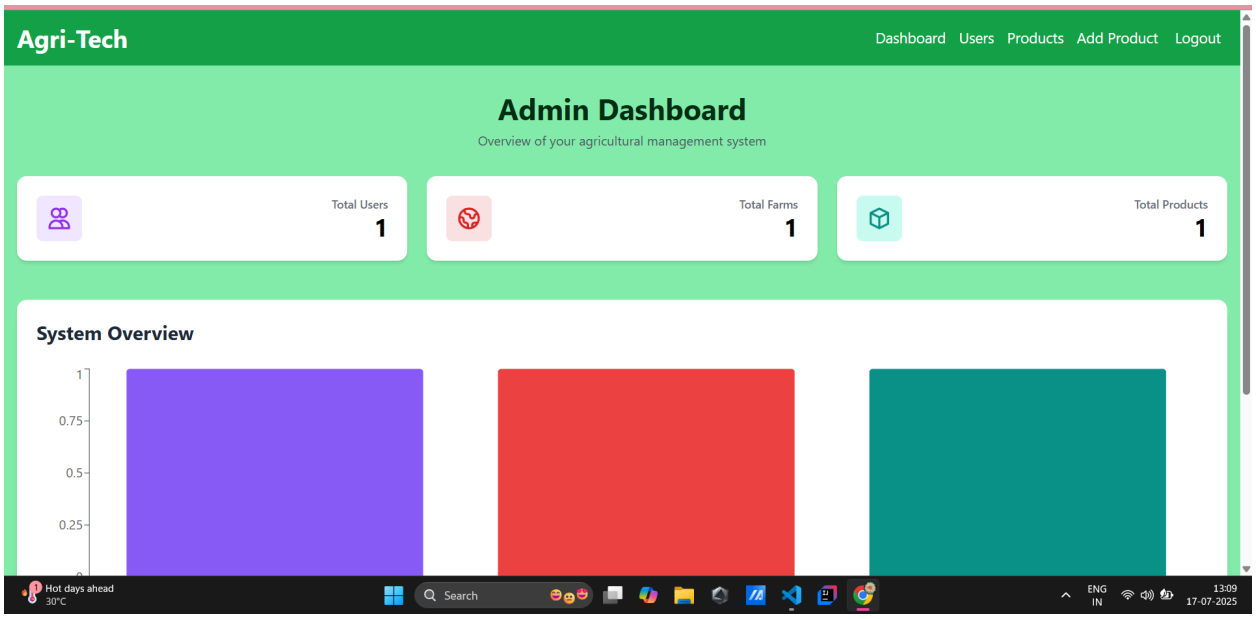
Add Crop:

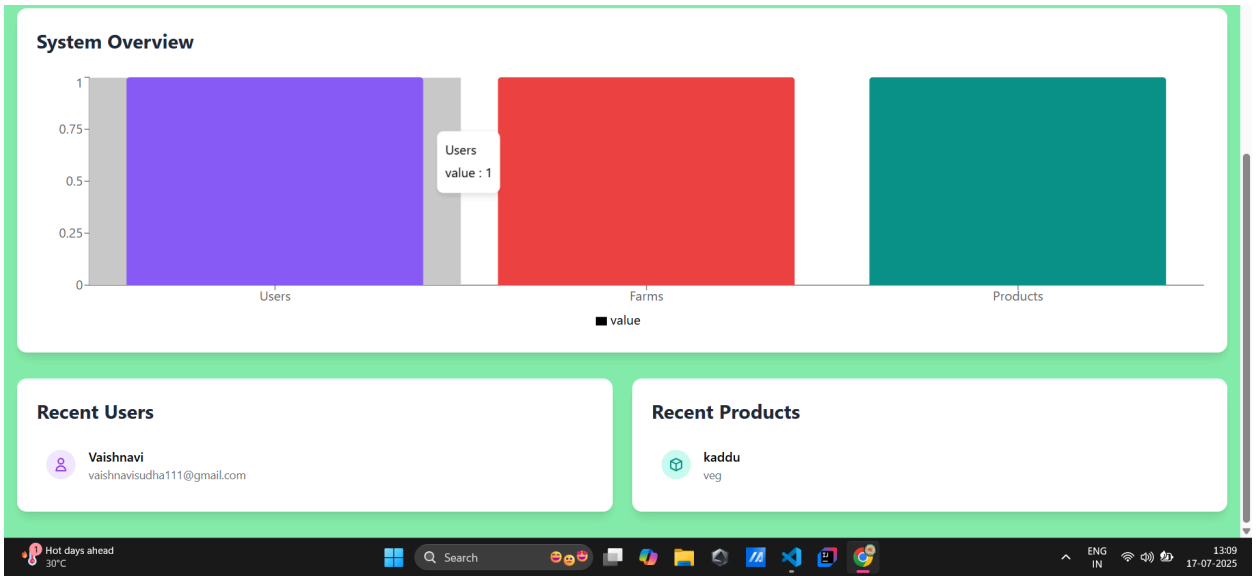


My Orders:

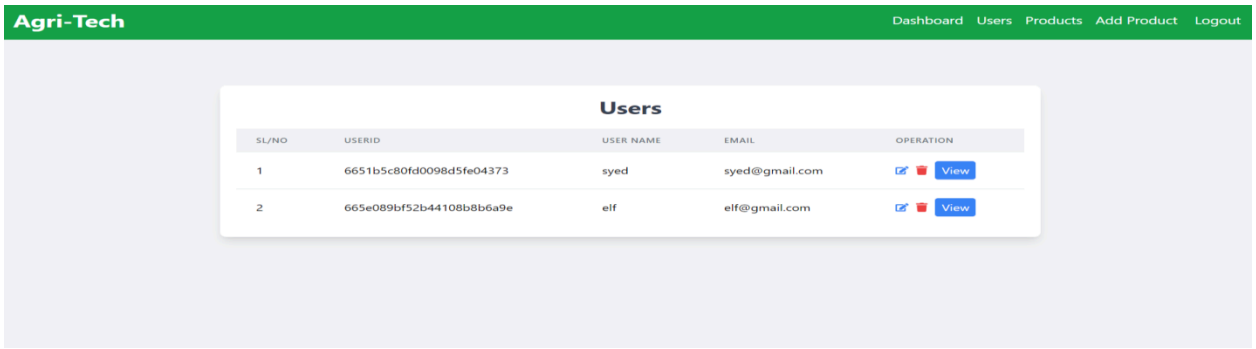


Admin Dashboard:

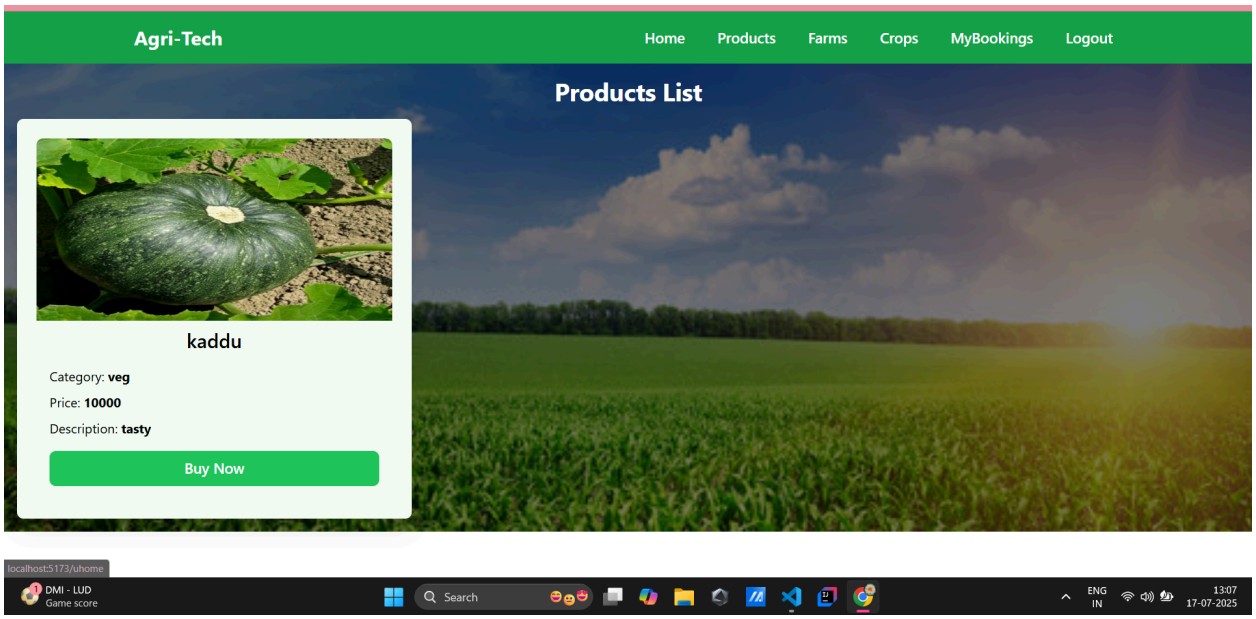

Agri-Tech
Dashboard Users Products Add Product Logout
Admin Dashboard
Overview of your agricultural management system
Total Users
1
Total Farms
1
Total Products
1
System Overview
1
0.75
0.5
0.25
Hot days ahead
30°C
Search
ENG
IN
13:09
17-07-2025


System Overview
1
0.75
0.5
0.25
0
Users
value : 1
Users
Farms
Products
value
Recent Users
Vaishnavi
vaishnavisudha111@gmail.com
Recent Products
kaddu
veg
Hot days ahead
30°C
Search
ENG
IN
13:09
17-07-2025

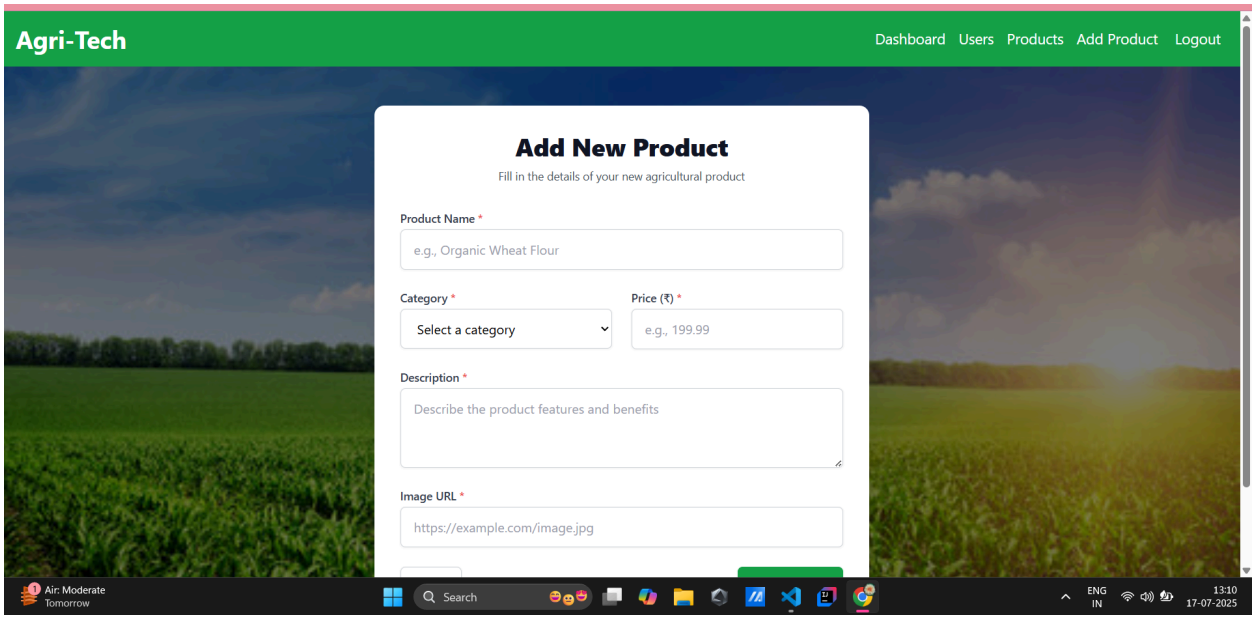
Users:


Agri-Tech
Dashboard Users Products Add Product Logout
Users
SL/NO
USERID
USER NAME
EMAIL
OPERATION
1
6651b5c80fd0098d5fe04373
syed
syed@gmail.com
View
2
665e089bf52b44108b8b6a9e
elf
elf@gmail.com
View

Products:



Add Product



Reference Link :-

Demo Videos :- Demo Videos

Code : code

***** Happy Hacking!! *****